



Laravel

Afrizal Doewes

Top 7 Popular PHP Frameworks

To Consider For Your Next Project



Why PHP Frameworks?

- **Expressive Syntax:** Laravel offers an elegant syntax and powerful features for web application development.
- **Progressive Framework:** From novice to expert, Laravel provides comprehensive resources to support your growth.
- **Scalability:** Scale your applications effortlessly with PHP and Laravel's support for distributed cache systems.
- **Community Support:** Benefit from the collective knowledge of thousands of developers contributing to Laravel's ecosystem.

Introduction to Laravel

- Laravel is a web application framework known for its expressive syntax and powerful features.
- It provides a structure and starting point for creating modern, full-stack web applications, allowing developers to focus on building amazing projects while handling intricate details effortlessly.



Creating a Laravel project

- Before creating the first Laravel project, ensure the local machine has **PHP** and **Composer** installed.
- On macOS or Windows, install PHP and Composer via Laravel Herd, making setup a breeze.
- Additionally, Node and NPM are recommended.

Creating a Laravel project

- A new Laravel project can be created using Composer's **create-project** command:

```
composer create-project laravel/laravel example-app
```

- Or, by globally installing the Laravel installer via Composer:

```
composer global require laravel/installer
```

```
laravel new example-app
```

Creating a Laravel project

- Once the project is created, launch Laravel's local development server with Artisan's serve command:

cd example-app

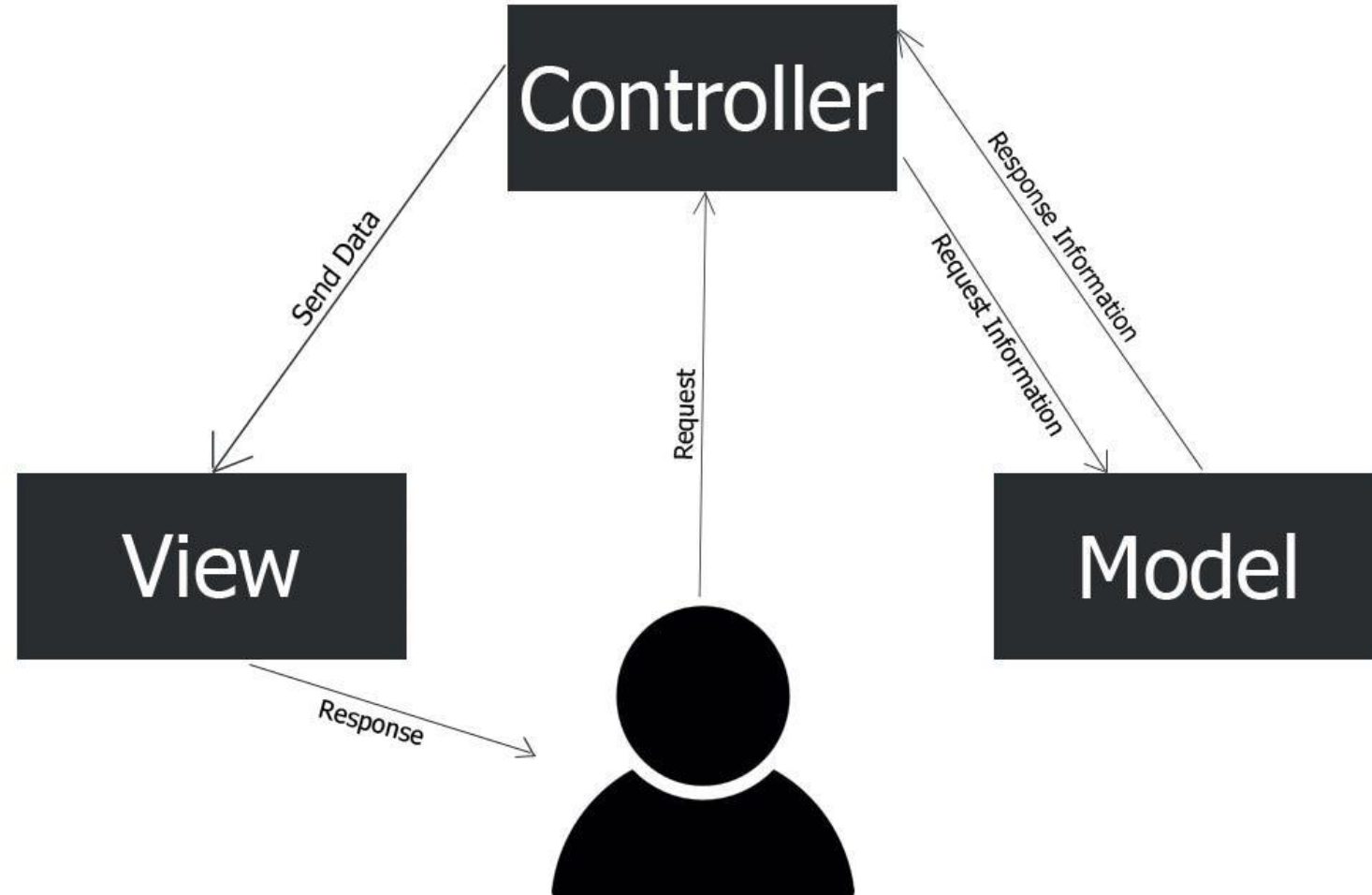
php artisan serve

- The application will be accessible in the browser at <http://localhost:8000> .

MVC (Model-View-Controller) Architecture

- Laravel is a popular PHP web application framework that follows the Model-View-Controller (MVC) architectural pattern.
- The MVC pattern is a software design pattern that separates an application into three interconnected components: the model, the view, and the controller.

Model-View-Controller



Make a Controller

- To create a controller, use this command:

```
php artisan make:controller HomeController
```

- Then, create the route in web.php:

```
Route::get('/home', [HomeController::class, 'index']);
```

- Import controller in web.php:

```
use App\Http\Controllers\HomeController;
```

Model & Database

- To create a model, use this command:

php artisan make:model Student -m

- Modify the function 'up' in the migration file:

```
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name')->nullable();
        $table->string('email')->nullable();
        $table->string('image')->nullable();
        $table->timestamps();
    });
}
```

- Then, continue with the database migration:

php artisan migrate

Insert Data to Database

- Create form to upload data:

```
<form action="{{('upload')}}" method="POST" enctype="multipart/form-data">
```

```
@csrf
```

```
<div style="padding: 10px;">
```

```
<label>Name</label>
```

```
<input type="text" name="name">
```

```
</div>
```

```
</form>
```

- Create the route “upload” with method POST in web.php
- Then, create the corresponding function in the controller.

Create “upload” function in Controller

```
public function upload(Request $request){  
    $student = new student;
```

```
    $student->name = $request->name;  
    $student->email = $request->email;
```

```
    $image = $request->file;
```

```
    if($image){  
        $imagename = time() . '.' . $image->getClientOriginalExtension();  
        $request->file->move('student', $imagename);  
        $student->image = $imagename;  
    }
```

```
    $student->save();
```

```
    return redirect()->back();  
}
```

- Import model Student in HomeController:

```
use App\Models\student;
```

How to display the data in database

- To create a route to “view”
- Create the corresponding “view” controller
- Create the view.blade.php file in resources

```
@foreach($data as $student)
<tr>
    <td>{{ $student->name }}</td>
    <td>{{ $student->email }}</td>
    <td>
        
    </td>
    <td>
        <a href="{{ url('delete', $student->id) }}">Delete</a>
    </td>
</tr>
@endforeach
```

How to delete the data in database

- Create delete link using: `<a href="{{ url('delete', $student->id) }}">Delete</a>`

- To create a route to “delete” in web.php:

```
Route::get('/delete/{id}', [HomeController::class, 'delete']);
```

- Create the corresponding “delete” controller

```
public function delete($id){  
  
    $data = student::find($id);  
  
    $data->delete();  
  
    return redirect()->back();  
}
```

How to search for data in database

- Create form for “search”:

```
<form action="{{ ('search') }}" method="GET">
    @csrf
    <div style="padding: 10px;">
        <input type="text" name="search"
placeholder="Search for student">
        <input type="submit" value="Search">
    </div>
</form>
```

How to search for data in database

- To create a route to “search” in web.php:

```
Route::get('/search', [HomeController::class, 'search']);
```

- Create the corresponding “search” controller

```
public function search(Request $request) {  
    $search = $request->search;
```

```
    $data = student::where('name', 'like', '%' . $search . '%')->orWhere  
        ('email', 'like', '%' . $search . '%')->get();
```

```
    return view('view', compact('data'));  
}
```

How to update data in database

- Create form for “update” in a new view:

```
<form action="{{ url('update', $student->id) }}" method="POST"
enctype="multipart/form-data">
```

```
@csrf
```

```
<div style="padding: 10px;">
```

```
<label>Name</label>
```

```
<input type="text" name="name" value="{{ $student->name }}">
```

```
</div>
```

```
</form>
```

How to update data in database

- To create a route to “search” in web.php:

```
Route::get('/update_view/{id}', [HomeController::class, 'update_view']);  
Route::post('/update/{id}', [HomeController::class, 'update']);
```

- Create the corresponding “search” controller

```
public function update(Request $request, $id){  
  
    $student = student::find($id);  
  
    $student->name = $request->name;  
    $student->email = $request->email;  
  
    //the rest is the same as upload data ...  
}
```